

Legacy Goods Deactivation

Trovemo (litemall) · 2026-07-20 · applied to **dev** and **production (trovemo.com)** · commit `87bf78ac0`

1. Why

The catalog contained two kinds of goods:

	CJ dropshipping goods	Legacy goods
Count (on sale)	9,110	238 (+1 already off sale)
Identified by	<code>cj_pid</code> set	<code>cj_pid IS NULL OR cj_pid = ''</code>
Images	CJ CDN via <code>/_cdn/</code> edge proxy	<code>yanxuan.nosdn.127.net</code> (China, plain HTTP)
Fulfilment	Real — CJ places and ships the order	None — no inventory exists anywhere

The legacy set is the original litemall demo catalog that came with the seed data. If a customer ordered one of those items, **nothing could be shipped**. They were therefore taken off sale. The two definitions were verified to coincide exactly (239 = 239, zero CJ rows matched) in both databases before running anything.

Deliberately deactivated, not deleted. The store is built so that locally-managed goods go through the same full flow as CJ goods (catalog, pricing, cart, checkout, fulfilment seams). That pipeline is untouched and stays working. When real local inventory is registered later, those goods are simply loaded and switched on sale — same process, nothing to rebuild.

2. What was done — data

One idempotent, reversible statement, committed to the repo as `doc/ops/2026-07-20-deactivate-legacy-goods.sql`:

```
UPDATE litemall_goods
SET    is_on_sale = 0, update_time = NOW()
WHERE  deleted = 0 AND is_on_sale = 1
      AND (cj_pid IS NULL OR cj_pid = '');    -- 238 rows on first run
```

Environment	Applied	Result
Development DB (workstation)	✓ 2026-07-20	9,110 on sale, 0 legacy on sale
Production DB (trovemo.com VPS)	✓ 2026-07-20	9,110 on sale, 0 legacy on sale

Production search index (Elasticsearch)	✓ reindexed	9,110 documents (was 9,348)
Development search index	—	dev stack was not running; next dev reindex picks the change up automatically

3. What was done — code (the gap the data change alone could not close)

While verifying, a real gap surfaced: **nothing in the system prevented an off-sale good from being ordered**. The product page still rendered a working “Add to cart” button, the cart API accepted the item, and order submission validated only stock and price. That gap affected any good an admin takes off sale, not just the legacy set. Four changes close it end to end (all server-authoritative, in the Wave-7 spirit):

Layer	File	Change
Goods facade (order service)	<code>LiteMallGoodsFacadeImpl.mapGoods</code>	Maps the <code>onSale</code> flag from goods-management (it was never mapped, defaulting to <i>false</i>). A missing field maps to <i>true</i> so an older payload cannot block every purchase.
Cart add	<code>LiteMallOrderOrchestratorService.addToCart</code>	Rejects off-sale goods with a typed error: “... <i>is not available for sale</i> ”.
Order submit	<code>LiteMallOrderDomainService.validateProductStock</code>	Re-checks sale status against the live catalog for every cart line — a line added <i>before</i> a good was deactivated is caught here and produces a clean “no longer available, remove it from your cart” error instead of an unfulfillable order.
Product page (customer SPA)	<code>modules/product/Detail.tsx</code>	Off-sale goods show “ Currently unavailable ”; Add-to-cart / Buy-now are disabled. The page stays reachable by direct URL (old links don’t 404), it just cannot sell.

4. Verification (production)

- Storefront lists & search: 0 yanxuan-image goods across sampled pages; catalog serves HTTP 200.

- Search index: 9,110 documents indexed — exactly the CJ catalog.
- Off-sale detail page (e.g. goods 1006002): API returns `onSale: false`, and the deployed product-page bundle contains the “Currently unavailable” state (verified in the live JS chunk).
- API-level add-to-cart of the off-sale good 1006002: **rejected server-side, no cart row persisted** (tested live with a throwaway account, since removed; a CJ good added in the same session succeeded at its server-resolved price). The HTTP envelope is the module’s standard `errno 502`; the descriptive reason (“*...is not available for sale*”) appears in the order-service log, and the customer-friendly “no longer available — remove it from your cart” message surfaces at order submit.

5. How to bring goods back later (real local inventory)

1. Load / register the goods as usual through the admin console (or reuse the existing rows: `UPDATE litemall_goods SET is_on_sale = 1 WHERE id = <id>;` — full rollback statement is in the SQL file header).
2. Reindex: `./litemall-prod.sh reindex` — search serves from Elasticsearch, so without this step storefront visibility does not change.
3. Nothing else. The on-sale gates added above respect any good with `is_on_sale = 1`; local goods flow through cart → checkout → payment exactly like CJ goods.

Operational side-effect: `litemall-prod.sh smoke-checkout` deliberately tests the money path with a *local* (non-CJ) SKU. With every local good off sale it now stops with a clear message. To run it: re-activate one legacy good, reindex, test, revert (the script prints these exact steps).

6. References

- SQL (idempotent, with rollback): `doc/ops/2026-07-20-deactivate-legacy-goods.sql`
- Code + docs commit: `87bf78ac0` — “Deactivate legacy (non-CJ) goods + enforce on-sale at cart-add, submit, and PDP”
- Related same-day work: Cloudflare edge caching & DNS-01 (`d5fdb3c99...d7a4dc0d3`), which resolved the slow goods loading.